Enhancing B language reasoners with SMT techniques

Vincent Trélat

Université de Lorraine, CNRS, Inria, Loria, Nancy, France









Outline

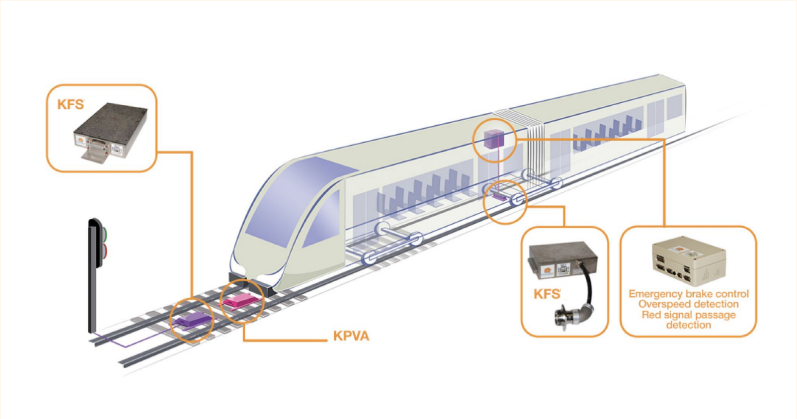
1. **Introduction**
2. **Encoding B proof obligations in SMT-LIB using HOL**
   - Encoding sets
   - Encoding functions
   - Example
3. **Semantics**
4. **A more elaborate / realistic example**
5. **Conclusion**

Context

Formal methods for safety-critical systems, e.g. railways

Context

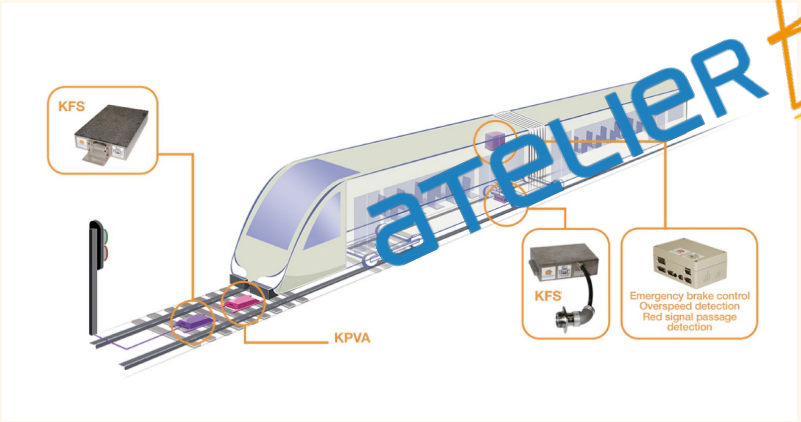
Formal methods for safety-critical systems, e.g. railways



CLEARSY
Safety Solutions Designer

Context

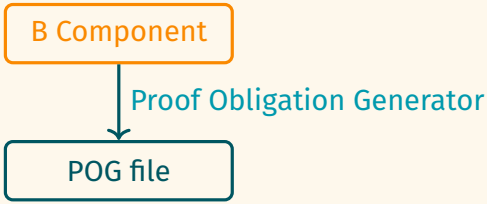
Formal methods for safety-critical systems, e.g. railways



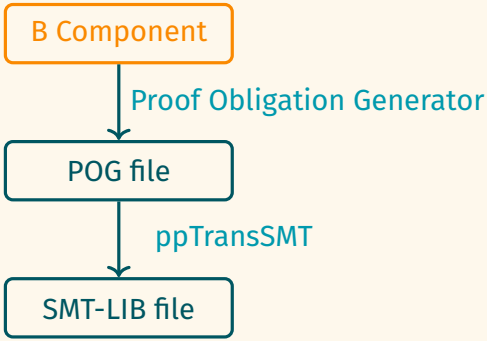
Introduction

B Component

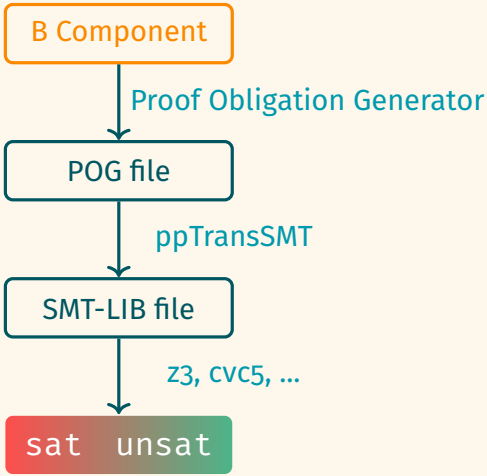
Introduction



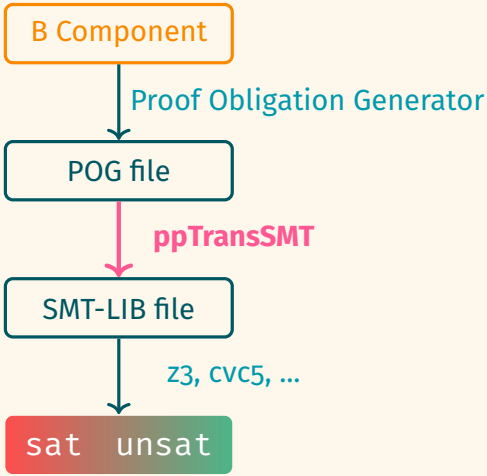
Introduction



Introduction



Introduction



B and Atelier B

B

- Formal method for software and hardware development
- Based on **ZFC**
- Structured around abstract machines, variables, invariants, and operations.

B and Atelier B

B

- Formal method for software and hardware development
- Based on **ZFC**
- Structured around abstract machines, variables, invariants, and operations.

Atelier B

- Suite of tools for B development
- Includes a proof obligation generator and a (FO) predicate prover
- Tries to automatically discharge proof obligations

SMT-LIB

- Standard input format for SMT solvers (e.g. z3, cvc5, veriT)
- Based on many-sorted first-order logic (officially)
- Comes with many theories (e.g. arrays, integer and real linear arithmetic)

1. Introduction

2. **Encoding B proof obligations in SMT-LIB using HOL**
   - Encoding sets
   - Encoding functions
   - Example

3. Semantics

4. A more elaborate / realistic example

5. Conclusion

Current encoding (ppTrans)

A **first-order** encoding of sets:

Current encoding (ppTrans)

A **first-order** encoding of sets:

- Based on FOL with equality and uninterpreted functions (EUF)

Current encoding (ppTrans)

A **first-order** encoding of sets:

- Based on FOL with equality and uninterpreted functions (EUF)
- Sets are only *specified* through the use of an uninterpreted predicate $\in$

Current encoding (ppTrans)

A **first-order** encoding of sets:

- Based on FOL with equality and uninterpreted functions (EUF)
- Sets are only *specified* through the use of an uninterpreted predicate $\in$
- Only expressions like $x \in S$ are encoded

Current encoding (ppTrans)

A **first-order** encoding of sets:

- Based on FOL with equality and uninterpreted functions (EUF)
- Sets are only *specified* through the use of an uninterpreted predicate $\in$
- Only expressions like $x \in S$ are encoded

```
SETS
  S = {e1, e2, e3}
```

Current encoding (ppTrans)

A **first-order** encoding of sets:

- Based on FOL with equality and uninterpreted functions (EUF)
- Sets are only *specified* through the use of an uninterpreted predicate $\in$
- Only expressions like $x \in S$ are encoded

```
SETS
  S = {e1, e2, e3}
```

$\rightarrow$

```
(declare-fun S () (P Int))
(declare-fun e1 () Int)
(declare-fun e2 () Int)
(declare-fun e3 () Int)
(declare-fun ∈₀ ((Int) (P Int)) Bool)

(assert (forall ((x Int)) (=
  (∈₀ x S)
  (or (= x e1) (= x e2) (= x e3)))))
```

Suggested encoding

A **higher-order** encoding of sets:

Suggested encoding

A **higher-order** encoding of sets:

- Uses some extensions of SMT-LIB 2.6 to HOL brought by cvc5 (λ-abstraction, arrow type constructor)

Suggested encoding

A **higher-order** encoding of sets:

- Uses some extensions of SMT-LIB 2.6 to HOL brought by cvc5 (λ-abstraction, arrow type constructor)
- Sets are represented by their characteristic predicate: no need for membership predicate!

Suggested encoding

A **higher-order** encoding of sets:

- Uses some extensions of SMT-LIB 2.6 to HOL brought by cvc5 (λ-abstraction, arrow type constructor)
- Sets are represented by their characteristic predicate: no need for membership predicate!

SETS
`S = {e1, e2, e3}`

Suggested encoding

A **higher-order** encoding of sets:

- Uses some extensions of SMT-LIB 2.6 to HOL brought by cvc5 (λ-abstraction, arrow type constructor)
- Sets are represented by their characteristic predicate: no need for membership predicate!

```
SETS
  S = {e1, e2, e3}
```



```
(declare-const e1 Int)
(declare-const e2 Int)
(declare-const e3 Int)
(define-const S (→ Int Bool) (lambda
   ((x Int))
  (or (= x e1) (= x e2) (= x e3))))
```

Currently, functions such as $f \in A \pfun B$ are represented by:

- the set $\mathcal{P}(A \times B)$
- stating that the relation is functional[1]

$$\forall x\, y\, z.\, x \mapsto y \in f \land x \mapsto z \in f \Rightarrow y = z \quad \text{(functionality)}$$

- additional properties of f (totality, partiality, injectivity...)

$$\forall x_1\, y_1\, x_2\, y_2.\, x_1 \mapsto y_1 \in f \land x_2 \mapsto y_2 \in f \land y_1 = y_2 \Rightarrow x_1 = x_2 \quad \text{(injectivity)}$$

$$\vdots$$

[1] Read $x \mapsto y \in f$ as $f(x) = y$

Currently, functions such as $f \in A \nrightarrow B$ are represented by:

- the set $\mathcal{P}(A \times B)$
- stating that the relation is functional[1]

$$\forall x\, y\, z.\, x \mapsto y \in f \land x \mapsto z \in f \Rightarrow y = z \quad \text{(functionality)}$$

- additional properties of f (totality, partiality, injectivity...)

$$\forall x_1\, y_1\, x_2\, y_2.\, x_1 \mapsto y_1 \in f \land x_2 \mapsto y_2 \in f \land y_1 = y_2 \Rightarrow x_1 = x_2 \quad \text{(injectivity)}$$

$$\vdots$$

Can we find a better way to encode functions in order to preserve their properties and avoid additional overhead?

[1] Read $x \mapsto y \in f$ as $f(x) = y$

Currently, functions such as $f \in A \nrightarrow B$ are represented by:

- the set $\mathcal{P}(A \times B)$
- **stating that the relation is functional**[1]

$$\forall x\, y\, z.\, x \mapsto y \in f \land x \mapsto z \in f \Rightarrow y = z \quad \text{(functionality)}$$

- additional properties of f (totality, partiality, injectivity...)

$$\forall x_1\, y_1\, x_2\, y_2.\, x_1 \mapsto y_1 \in f \land x_2 \mapsto y_2 \in f \land y_1 = y_2 \Rightarrow x_1 = x_2 \quad \text{(injectivity)}$$

$$\vdots$$

Can we find a better way to encode functions in order to preserve their properties and avoid **additional overhead**?

[1] Read $x \mapsto y \in f$ as $f(x) = y$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\forall a : \mathtt{int} \cdot \exists b : \mathtt{int}, f : \mathtt{set}\,(\tau \times \mathtt{int}) \cdot f \in S \rightarrowtail a..b$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\forall a : \mathtt{int} \cdot \exists b : \mathtt{int}, f : \mathtt{set}\,(\tau \times \mathtt{int}) \cdot f \in S \rightarrowtail a..b$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\forall a : \mathtt{int} \cdot \exists b : \mathtt{int}, f : \mathtt{set}(\tau \times \mathtt{int}) \cdot f \in S \pfun a..b \land S \subseteq \mathbf{dom}(f) \land _\mathtt{inj}(f)$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\forall a\colon \mathtt{int} \cdot \exists b\colon \mathtt{int}, f\colon \mathtt{set}\,(\tau \times \mathtt{int}) \cdot f \in S \pfun a..b \land S \subseteq \mathbf{dom}(f) \land _\mathtt{inj}(f)$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\forall a\colon \mathtt{int}\cdot\exists b\colon \mathtt{int}, f\colon \mathtt{set}\,(\tau\times\mathtt{int})\cdot f \in S \leftrightarrow a..b \wedge _\mathtt{func}(f) \wedge S \subseteq \mathbf{dom}(f) \wedge _\mathtt{inj}(f)$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\forall a\colon \mathtt{int}\cdot\exists b\colon \mathtt{int}, f\colon \mathtt{set}\,(\tau\times\mathtt{int})\cdot f \in S \leftrightarrow a..b \wedge _\mathtt{func}(f) \wedge S \subseteq \mathbf{dom}(f) \wedge _\mathtt{inj}(f)$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a\colon \mathtt{int} \cdot \exists b\colon \mathtt{int}, f\colon \mathtt{set}(\tau \times \mathtt{int}) \cdot \\
&\qquad f \in S \leftrightarrow a..b && \wedge \\
&\qquad \forall x\colon \tau, y\colon \tau, z\colon \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z && \wedge \\
&\qquad S \subseteq \mathbf{dom}(f) && \wedge \\
&\qquad _\mathtt{inj}(f)
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a\colon \mathtt{int} \cdot \exists b\colon \mathtt{int}, f\colon \mathtt{set}\,(\tau \times \mathtt{int}) \cdot \\
&\qquad f \in S \leftrightarrow a..b && \wedge \\
&\qquad \forall x\colon \tau, y\colon \mathtt{int}, z\colon \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z && \wedge \\
&\qquad S \subseteq \mathbf{dom}(f) && \wedge \\
&\qquad _\mathtt{inj}(f)
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a\colon \mathtt{int} \cdot \exists b\colon \mathtt{int}, f\colon \mathtt{set}(\tau \times \mathtt{int}) \cdot \\
&\qquad \forall x\colon \tau, y\colon \mathtt{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b && \wedge \\
&\qquad \forall x\colon \tau, y\colon \tau, z\colon \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z && \wedge \\
&\qquad S \subseteq \mathbf{dom}(f) && \wedge \\
&\qquad _\mathtt{inj}(f)
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a : \mathtt{int} \cdot \exists b : \mathtt{int}, f : \mathtt{set}(\tau \times \mathtt{int}) \cdot \\
&\qquad \forall x : \tau, y : \mathtt{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b && \wedge \\
&\qquad \forall x : \tau, y : \mathtt{int}, z : \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z && \wedge \\
&\qquad S \subseteq \mathbf{dom}(f) && \wedge \\
&\qquad _\mathtt{inj}(f)
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a : \mathtt{int} \cdot \exists b : \mathtt{int}, f : \mathtt{set}(\tau \times \mathtt{int}) \cdot \\
&\qquad \forall x : \tau, y : \mathtt{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b && \wedge \\
&\qquad \forall x : \tau, y : \mathtt{int}, z : \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z && \wedge \\
&\qquad \forall z : \tau \cdot z \in S \Rightarrow \exists w : \mathtt{int} \cdot z \mapsto w \in f && \wedge \\
&\qquad _\mathtt{inj}(f)
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a : \mathtt{int} \cdot \exists b : \mathtt{int}, f : \mathtt{set}(\tau \times \mathtt{int}) \cdot \\
&\qquad \forall x : \tau, y : \mathtt{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b \qquad \wedge \\
&\qquad \forall x : \tau, y : \mathtt{int}, z : \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z \qquad \wedge \\
&\qquad \forall z : \tau \cdot z \in S \Rightarrow \exists w : \mathtt{int} \cdot z \mapsto w \in f \qquad \wedge \\
&\qquad _\mathtt{inj}(f)
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a\colon \mathtt{int} \cdot \exists b\colon \mathtt{int}, f\colon \mathtt{set}\,(\tau \times \mathtt{int}) \cdot \\
&\qquad \forall x\colon \tau, y\colon \mathtt{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b && \wedge \\
&\qquad \forall x\colon \tau, y\colon \mathtt{int}, z\colon \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z && \wedge \\
&\qquad \forall z\colon \tau \cdot z \in S \Rightarrow \exists w\colon \mathtt{int} \cdot z \mapsto w \in f && \wedge \\
&\qquad \forall x\colon \tau, y\colon \tau, z\colon \tau \cdot x \mapsto z \in f \wedge y \mapsto z \in f \Rightarrow x = y
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$
\begin{aligned}
&\forall a\colon \mathtt{int} \cdot \exists b\colon \mathtt{int}, f\colon \mathtt{set}\,(\tau \times \mathtt{int}) \cdot \\
&\qquad \forall x\colon \tau, y\colon \mathtt{int} \cdot x \mapsto y \in f \Rightarrow x \in S \wedge a \leq y \wedge y \leq b \qquad \wedge \\
&\qquad \forall x\colon \tau, y\colon \mathtt{int}, z\colon \mathtt{int} \cdot x \mapsto y \in f \wedge x \mapsto z \in f \Rightarrow y = z \quad \wedge \\
&\qquad \forall z\colon \tau \cdot z \in S \Rightarrow \exists w\colon \mathtt{int} \cdot z \mapsto w \in f \qquad \wedge \\
&\qquad \forall x\colon \tau, y\colon \tau, z\colon \tau \cdot x \mapsto z \in f \wedge y \mapsto z \in f \Rightarrow x = y
\end{aligned}
$$

How large is the overhead?

Let S be a set of elements of type τ. The expression **finite** S is translated as follows:

$$\exists N : \mathtt{int}, f : \tau \to \mathtt{int} \cdot$$
$$\forall x : \tau, y : \tau, z : \mathtt{int} \cdot f(x) = z \wedge f(y) = z \Rightarrow x = y \quad \wedge$$
$$\forall x : \tau \cdot x \in S \Rightarrow 0 \leq f(x) \wedge f(x) < N$$

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$.

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$. Let op be the following operation:

```
op (x, y) =
PRE
  x : INTEGER & y : INTEGER  // x ∈ ℤ ∧ y ∈ ℤ
THEN
  f := f <+ {x |-> y}        // f := f ∪ {x ↦ y}
END
```

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$. Let op be the following operation:

```
op (x, y) =
PRE
  x : INTEGER & y : INTEGER  // x ∈ ℤ ∧ y ∈ ℤ
THEN
  f := f <+ {x |-> y}        // f := f ∪ {x ↦ y}
END
```

Assume $f := \{0 \mapsto 1, 1 \mapsto 0\}$.

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$. Let op be the following operation:

```
op (x, y) =
PRE
  x : INTEGER & y : INTEGER  // x ∈ ℤ ∧ y ∈ ℤ
THEN
  f := f  ∨  {x |-> y}          // f := f ∪ {x ↦ y}
END
```

Assume $f := \{0 \mapsto 1, 1 \mapsto 0\}$.

- After $\mathsf{op}(2,3)$, $f = \{0 \mapsto 1, 1 \mapsto 0, 2 \mapsto 3\}$

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$. Let op be the following operation:

```
op (x, y) =
PRE
  x : INTEGER & y : INTEGER  // x ∈ ℤ ∧ y ∈ ℤ
THEN
  f := f ∨ {x |-> y}          // f := f ∪ {x ↦ y}
END
```

Assume $f := \{0 \mapsto 1, 1 \mapsto 0\}$.

- After $\mathsf{op}(2,3)$, $f = \{0 \mapsto 1, 1 \mapsto 0, 2 \mapsto 3\}$ ✓ (still a function)

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$. Let op be the following operation:

```
op (x, y) =
PRE
  x : INTEGER & y : INTEGER  // x ∈ ℤ ∧ y ∈ ℤ
THEN
  f := f ∨ {x |-> y}          // f := f ∪ {x ↦ y}
END
```

Assume $f := \{0 \mapsto 1, 1 \mapsto 0\}$.

- After $\mathsf{op}(2,3)$, $f = \{0 \mapsto 1, 1 \mapsto 0, 2 \mapsto 3\}$ ✓ (still a function)
- After $\mathsf{op}(0,2)$, $f = \{0 \mapsto 1, 1 \mapsto 0, 0 \mapsto 2\}$

A warning example

Let $f \in \mathbb{Z} \nrightarrow \mathbb{Z}$. Let op be the following operation:

```
op (x, y) =
PRE
  x : INTEGER & y : INTEGER  // x ∈ ℤ ∧ y ∈ ℤ
THEN
  f := f ∨ {x |-> y}          // f := f ∪ {x ↦ y}
END
```

Assume $f := \{0 \mapsto 1, 1 \mapsto 0\}$.

- After $\mathsf{op}(2,3)$, $f = \{0 \mapsto 1, 1 \mapsto 0, 2 \mapsto 3\}$ ✓ (still a function)
- After $\mathsf{op}(0,2)$, $f = \{0 \mapsto 1, 1 \mapsto 0, 0 \mapsto 2\}$ ✗ (no longer a function)

Set-lifting / Type inference

Definition (Set-lifting)

Set-lifting is denoted by $\uparrow\cdot$ and is defined inductively as follows:

- $\uparrow(A \times B) = \uparrow A \times \uparrow B$
- $\uparrow\mathcal{P}(A) = \mathcal{P}(\uparrow A)$
- $\uparrow\{x \in A \mid P\} = \uparrow A$
- $\uparrow\mathbb{B} = \mathbb{B}$, $\uparrow\mathbb{Z} = \mathbb{Z}$

Notation

$\cdot^?$ denotes the `Option` type.

Idea of the encoding for functions:

Rule: partial function

$f \in A \pfun B$ is encoded as:

Idea of the encoding for functions:

Rule: partial function

$f \in A \nrightarrow B$ is encoded as:

- $f \in \uparrow A \rightarrow \uparrow B^?$ (typing)

Idea of the encoding for functions:

Rule: partial function

$f \in A \nrightarrow B$ is encoded as:

- $f \in \uparrow A \rightarrow \uparrow B^?$ (typing)
- $\forall x \in \uparrow A.\ f\ x \neq \text{none} \Rightarrow x \in A$ (domain)

Idea of the encoding for functions:

Rule: partial function

$f \in A \nrightarrow B$ is encoded as:

- $f \in \uparrow A \rightarrow \uparrow B^?$ (typing)
- $\forall x \in \uparrow A.\ f\ x \neq \text{none} \Rightarrow x \in A$ (domain)
- $\forall x \in \uparrow A.\ f\ x \neq \text{none} \Rightarrow \exists y \in B.\ f\ x = \text{some}\ y$ (range)

Idea of the encoding for functions:

Rule: total function

$f \in A \to B$ is encoded as:

- $f \in \uparrow A \to \uparrow B^?$ (typing)
- $\forall x \in \uparrow A.\ f\ x \neq \text{none} \Leftrightarrow x \in A$ (domain)
- $\forall x \in \uparrow A.\ f\ x \neq \text{none} \Rightarrow \exists y \in B.\ f\ x = \text{some}\ y$ (range)

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \uparrow\mathbb{N} \to \uparrow a..b^? \\ \forall x \in \uparrow\mathbb{N}.\ f\ x \neq \mathsf{none} \Leftrightarrow x \in \mathbb{N} \\ \forall x \in \uparrow\mathbb{N}.\ f\ x \neq \mathsf{none} \Rightarrow \exists y \in a..b.\ f\ x = \mathsf{some}\ y \end{cases}$$

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \mathbb{Z} \to \mathbb{Z}^? \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Leftrightarrow x \in \mathbb{N} \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Rightarrow \exists y \in a..b.\, f\, x = \mathsf{some}\, y \end{cases}$$

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \mathbb{Z} \to \mathbb{Z}^? \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Leftrightarrow x \in \mathbb{N} \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Rightarrow \exists y \in a..b.\, f\, x = \mathsf{some}\, y \end{cases}$$

```
(declare-const f (→ Int (Option Int)))
(assert (forall ((x Int)) (= (not (= (f x) none)) (≤ 0 x))))
(assert (forall ((x Int)) (⇒ (not (= (f x) none))
  (exists ((y Int)) (and (≤ a y) (≤ y b) (= (f x) (some y)))))
  ))
```

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \mathbb{Z} \to \mathbb{Z}^? \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Leftrightarrow x \in \mathbb{N} \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Rightarrow \exists y \in a..b.\, f\, x = \mathsf{some}\, y \end{cases}$$

```
(declare-const f (→ Int (Option Int)))
(assert (forall ((x Int)) (= (not (= (f x) none)) (≤ 0 x))))
(assert (forall ((x Int)) (⇒ (not (= (f x) none))
  (exists ((y Int)) (and (≤ a y) (≤ y b) (= (f x) (some y)))))
  ))
```

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \mathbb{Z} \to \mathbb{Z}^? \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Leftrightarrow 0 \leq x \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Rightarrow \exists y \in a..b.\, f\, x = \mathsf{some}\, y \end{cases}$$

```
(declare-const f (→ Int (Option Int)))
(assert (forall ((x Int)) (= (not (= (f x) none)) (≤ 0 x))))
(assert (forall ((x Int)) (⇒ (not (= (f x) none))
  (exists ((y Int)) (and (≤ a y) (≤ y b) (= (f x) (some y)))))
  ))
```

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \mathbb{Z} \to \mathbb{Z}^? \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Leftrightarrow 0 \leq x \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Rightarrow \exists y \in a..b.\, f\, x = \mathsf{some}\, y \end{cases}$$

```
(declare-const f (→ Int (Option Int)))
(assert (forall ((x Int)) (= (not (= (f x) none)) (≤ 0 x))))
(assert (forall ((x Int)) (⇒ (not (= (f x) none))
  (exists ((y Int)) (and (≤ a y) (≤ y b) (= (f x) (some y)))))
  ))
```

A simple example

$$f \in \mathbb{N} \to a..b \quad \hookrightarrow \quad \begin{cases} f \in \mathbb{Z} \to \mathbb{Z}^? \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Leftrightarrow 0 \leq x \\ \forall x \in \mathbb{Z}.\, f\, x \neq \mathsf{none} \Rightarrow \exists y \in \mathbb{Z}.\, a \leq y \wedge y \leq b \wedge f\, x = \mathsf{some}\, y \end{cases}$$

```
(declare-const f (→ Int (Option Int)))
(assert (forall ((x Int)) (= (not (= (f x) none)) (≤ 0 x))))
(assert (forall ((x Int)) (⇒ (not (= (f x) none))
  (exists ((y Int)) (and (≤ a y) (≤ y b) (= (f x) (some y)))))
  ))
```

A simple example

$f \in \mathbb{N} \to a..b$ is encoded as:

```
(declare-const f (→ Int (Option Int)))
(assert (forall ((x Int)) (= (not (= (f x) none)) (⩽ 0 x))))
(assert (forall ((x Int)) (⇒ (not (= (f x) none))
  (exists ((y Int)) (and
    (⩽ a y) (⩽ y b)
    (= (f x) (some y)))))))
```

1 Introduction

2 Encoding B proof obligations in SMT-LIB using HOL
- Encoding sets
- Encoding functions
- Example

3 Semantics

4 A more elaborate / realistic example

5 Conclusion

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

- ease **set-theoretical reasoning**

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

- ease **set-theoretical reasoning**
- be **untyped**

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

- ease **set-theoretical reasoning**
- be **untyped**

Search for a theory $\mathcal{T}_{SMT}$ for interpreting SMT expressions such that at least $\mathcal{T}_B \hookrightarrow \mathcal{T}_{SMT}$ (or $\mathcal{T}_B \cong \mathcal{T}_{SMT}$)

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

- ease **set-theoretical reasoning**
- be **untyped**

Search for a theory $\mathcal{T}_{SMT}$ for interpreting SMT expressions such that at least $\mathcal{T}_B \hookrightarrow \mathcal{T}_{SMT}$ (or $\mathcal{T}_B \cong \mathcal{T}_{SMT}$)

$$\begin{aligned} \mathcal{T}_B &:= \mathbf{ZFC} \\ \mathcal{T}_{SMT} &\overset{?}{\in} \{\mathbf{ZFC}, \mathbf{HOL}\} \end{aligned}$$

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

- ease **set-theoretical reasoning**
- be **untyped**

Search for a theory $\mathcal{T}_{SMT}$ for interpreting SMT expressions such that at least $\mathcal{T}_B \hookrightarrow \mathcal{T}_{SMT}$ (or $\mathcal{T}_B \cong \mathcal{T}_{SMT}$)

$$\left.\begin{array}{rl} \mathcal{T}_B & := \mathbf{ZFC} \\ \mathcal{T}_{SMT} & \overset{?}{\in} \{\mathbf{ZFC}, \mathbf{HOL}\} \end{array}\right\}$$

Is the encoding sound?

Search for a **suitable theory** $\mathcal{T}_B$ for interpreting B expressions.

A good candidate should:

- ease **set-theoretical reasoning**
- be **untyped**

Search for a theory $\mathcal{T}_{SMT}$ for interpreting SMT expressions such that at least $\mathcal{T}_B \hookrightarrow \mathcal{T}_{SMT}$ (or $\mathcal{T}_B \cong \mathcal{T}_{SMT}$)

$$\left.\begin{array}{ll} \mathcal{T}_B & := \textbf{ZFC} \\ \mathcal{T}_{SMT} & \overset{?}{\in} \{\textbf{ZFC}, \textbf{HOL}\} \end{array}\right\} \quad \textbf{denotational semantics}$$

Typing B

Definition (B types)

B types are defined inductively as follows:

$$\begin{aligned} &\mathit{BType} ::= \\ &\mid \mathit{int} \mid \mathit{bool} \mid \mathit{set}\ \alpha \mid \alpha \times \beta : \mathit{BType} \end{aligned}$$

Definition (Context)

A typing context Γ is a map from variables to types.

Typing rules

$$\frac{\Gamma(v) = \tau}{\Gamma \vdash v : \tau}\ \textit{var} \qquad \frac{\Gamma \vdash x : \alpha \qquad \Gamma \vdash y : \beta}{\Gamma \vdash x \mapsto y : \alpha \times \beta}\ \textit{maplet} \qquad \frac{\Gamma \vdash x : \alpha \qquad \Gamma \vdash S : \textit{set}\ \alpha}{\Gamma \vdash x \in S : \textit{bool}}\ \textit{mem}$$

Typing rules

$$\frac{\Gamma(v) = \tau}{\Gamma \vdash v : \tau}\ \textit{var} \qquad \frac{\Gamma \vdash x : \alpha \qquad \Gamma \vdash y : \beta}{\Gamma \vdash x \mapsto y : \alpha \times \beta}\ \textit{maplet} \qquad \frac{\Gamma \vdash x : \alpha \qquad \Gamma \vdash S : \textit{set}\ \alpha}{\Gamma \vdash x \in S : \textit{bool}}\ \textit{mem}$$

$$\frac{\Gamma \vdash D : \textit{set}\ \alpha \qquad \Gamma, v : \alpha \vdash P : \textit{bool}}{\Gamma \vdash \{x \in D \mid P\} : \textit{set}\ \alpha}\ \textit{collect}_1$$

$$\frac{\Gamma, x_0 : \alpha_0 \vdash \{x_1, \ldots, x_n \in E \mid P\} : \textit{set}\ (\alpha_1 \times \ldots \times \alpha_n) \qquad \Gamma \vdash D : \textit{set}\ \alpha_0}{\Gamma \vdash \{x_0, \ldots, x_n \in E \mid P\} : \textit{set}\ (\alpha_0 \times \ldots \times \alpha_n)}\ \textit{collect}$$

Definition (Denotational semantics for B)

For any B term t, we define

$$[\![t]\!] : (\exists \Gamma\, \tau,\ \Gamma \vdash t : \tau) \to \mathsf{Set}_{\mathsf{ZFC}}$$

as a total function: every well-typed term has a denotation.

Definition (Denotational semantics for B)

For any B term t, we define

$$[\![t]\!] : (\exists \Gamma\, \tau,\ \Gamma \vdash t : \tau) \to \mathsf{Set}_{\mathsf{ZFC}}$$

as a total function: every well-typed term has a denotation.

$$[\![v]\!] \langle \Gamma, \tau, \dots \rangle = \mathcal{U}_\tau$$

Definition (Denotational semantics for B)

For any B term t, we define

$$[\![t]\!] : (\exists \Gamma\, \tau,\ \Gamma \vdash t : \tau) \to \mathsf{Set}_{\mathsf{ZFC}}$$

as a total function: every well-typed term has a denotation.

$$[\![v]\!]\,\langle \Gamma, \tau, \dots \rangle = \mathcal{U}_\tau$$

$$[\![\forall x \in D \cdot P]\!]\,\langle \Gamma, \mathtt{Bool}, \dots \rangle = \bigcap_{[\![D]\!]\langle \Gamma, \mathtt{set}\ \tau, \dots \rangle} [\![P]\!]\,\langle \Gamma, \mathtt{Bool}, \dots \rangle$$

Definition (Denotational semantics for B)

For any B term t, we define

$$[\![t]\!] : (\exists \Gamma\, \tau,\ \Gamma \vdash t : \tau) \to \mathsf{Set}_{\mathsf{ZFC}}$$

as a total function: every well-typed term has a denotation.

$$[\![v]\!]\,\langle \Gamma, \tau, \dots \rangle = \mathcal{U}_\tau$$

$$[\![\forall x \in D \cdot P]\!]\,\langle \Gamma, \mathtt{Bool}, \dots \rangle = \bigcap_{[\![D]\!]\langle \Gamma, \mathtt{set}\ \tau, \dots \rangle} [\![P]\!]\,\langle \Gamma, \mathtt{Bool}, \dots \rangle$$

$$[\![A \to B]\!]\,\langle \Gamma, \mathtt{set}(\alpha \times \beta), \dots \rangle = ([\![B]\!]\,\langle \Gamma, \mathtt{set}\ \beta, \dots \rangle)^{[\![A]\!]\langle \Gamma, \mathtt{set}\ \alpha, \dots \rangle}$$

A more elaborate / realistic example

```
MACHINE
  M
VARIABLES
  s0
INVARIANT
  s0 ⊆ NAT ∧
  s0 ∩ (ℤ \ ℕ) ∈ FIN(ℤ)
INITIALISATION
  s0 :∈ 𝒫(NAT)
END
```

A more elaborate / realistic example

```
MACHINE
  M
VARIABLES
  s0
INVARIANT
  s0 ⊆ NAT ∧
  s0 ∩ (ℤ \ ℕ) ∈ FIN(ℤ)
INITIALISATION
  s0 :∈ 𝒫(NAT)
END
```

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\mathbf{NAT}) \Rightarrow s0 \subseteq \mathbf{NAT} \land s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \mathbf{FIN}(\mathbb{Z})$$

A more elaborate / realistic example

MACHINE
  M
VARIABLES
  s0
INVARIANT
  $s0 \subseteq \mathbf{NAT} \land$
  $s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \mathbf{FIN}(\mathbb{Z})$
INITIALISATION
  $s0 :\in \mathcal{P}(\mathbf{NAT})$
END

The following proof obligation is generated:

$$s0 \in \mathcal{P}(\mathbf{NAT}) \Rightarrow s0 \subseteq \mathbf{NAT} \land s0 \cap (\mathbb{Z} \setminus \mathbb{N}) \in \mathbf{FIN}(\mathbb{Z})$$

which boils down to proving:

$$s0 \in \mathcal{P}(\mathbf{NAT}) \Rightarrow s0 \in \mathbf{FIN}(\mathbb{Z})$$

A more elaborate / realistic example

```
MACHINE
  M
VARIABLES
  s0
INVARIANT
  s0 ⊆ NAT ∧
  s0 ∩ (ℤ \ ℕ) ∈ FIN(ℤ)
INITIALISATION
  s0 :∈ 𝒫(NAT)
END
```

✗ predicate prover from Atelier B

A more elaborate / realistic example

```
MACHINE
  M
VARIABLES
  s0
INVARIANT
  s0 ⊆ NAT ∧
  s0 ∩ (ℤ \ ℕ) ∈ FIN(ℤ)
INITIALISATION
  s0 :∈ 𝒫(NAT)
END
```

- ✗ predicate prover from Atelier B
- ✗ CVC5 with ppTransSMT's encoding

A more elaborate / realistic example

```
MACHINE
  M
VARIABLES
  s0
INVARIANT
  s0 ⊆ NAT ∧
  s0 ∩ (ℤ \ ℕ) ∈ FIN(ℤ)
INITIALISATION
  s0 :∈ ℙ(NAT)
END
```

✗ predicate prover from Atelier B

✗ CVC5 with ppTransSMT's encoding

✓ CVC5 with the **higher-order** encoding

Conclusion

- Leverage recent extensions to fragments of **HOL** in SMT-LIB to encode B proof obligations

Conclusion

- Leverage recent extensions to fragments of **HOL** in SMT-LIB to encode B proof obligations
- Needs extensive testing (against *ppTrans*).

Conclusion

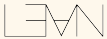
- Leverage recent extensions to fragments of **HOL** in SMT-LIB to encode B proof obligations
- Needs extensive testing (against *ppTrans*).
- Write the semantics of SMT-LIB

Conclusion

- Leverage recent extensions to fragments of **HOL** in SMT-LIB to encode B proof obligations
- Needs extensive testing (against *ppTrans*).
- Write the semantics of SMT-LIB
- **Prove soundness**

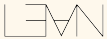
Conclusion

- Leverage recent extensions to fragments of **HOL** in SMT-LIB to encode B proof obligations
- Needs extensive testing (against *ppTrans*).
- Write the semantics of SMT-LIB
- **Prove soundness**



Conclusion

- Leverage recent extensions to fragments of **HOL** in SMT-LIB to encode B proof obligations
- Needs extensive testing (against *ppTrans*).
- Write the semantics of SMT-LIB
- **Prove soundness**



Questions?